

User Manual

ZKFinger SDK for Android

Record of Release

Date	Version	Description	Author
2015-11-15	1.0.0	Completed the basic version.	Chen Jianxing
2015-11-20	1.0.1	Added asynchronous interfaces.	Chen Jianxing
2015-12-10	1.0.3	Added some fuctions	Chen Jianxing
2019-06-10	1.0.4	Merge FingerprintService.class to zkandroidfingerservice.jar	Chen Jianxing
2019-10-22	1.0.5	Added 3.4.12~3.4.17	Chen Jianxing
2020-03-20	1.0.6	Add some functions and error code	Chen Jianxing

Important Claims

Thank you for using ZKTeco ZKFinger SDK. Please carefully read this document before using so that you can quickly grasp and use the ZKFinger SDK.

Copyright Statement

Without the written permission of the company, any organization or individual shall not extract or reproduce the entire or part of the content of the manual, or transmit the content in any form. This manual may contain the software whose copyright owner might be our company or other individuals. This software shall not be reproduced, distributed, revised, extracted, decompiled, disassembled, decoded, reverse engineered, rented, transferred, sublicensed, or infringed by other violations of copyright without the permission of the owner, but except for the actions allowed by laws.

Document Use Instructions

Due to constant expansion of ZKFinger SDK software functions, the version of the ZKFinger SDK document is updated continuously. Therefore, please carefully read the ZKFinger SDK document while using the ZKFinger SDK software. Please forgive us for the inconvenience brought to you. You can contact our document writer and the contact information is as follows.

Table of Contents

1. Overview.....	6
2. Development Environment Setup.....	6
3. ZKFinger SDK.....	9
3.1FingerprintSensor.class.....	11
3.1.1 open.....	12
3.1.2 close.....	12
3.1.3 retrieveLastImage.....	13
3.1.4 acquireTemplate.....	14
3.1.5 getLastTempLen.....	15
3.1.6 setFingerprintCaptureListener.....	15
3.1.7 setFingerprintCaptureMode.....	16
3.1.8 startCapture.....	17
3.1.9 stopCapture.....	17
3.1.10 Destroy.....	18
3.2 FingerprintCaptureListener.class.....	20
3.2.1 captureOK.....	20
3.2.2 captureError.....	21
3.3FingerprintSensorException.class.....	21
3.3.1 getErrorCode.....	21
3.3.2 getInternalErrorCode.....	22
3.4 ZKFingerService.class.....	22
3.4.1 init.....	22
3.4.2 free.....	22
3.4.3 verify.....	23
3.4.4 verifyId.....	24
3.4.5 identify.....	25
3.4.6 merge.....	27

3.4.7 save.....	28
3.4.8 get.....	29
3.4.9 del.....	30
3.4.10 clear.....	30
3.4.11 count.....	31
3.4.12 version.....	31
3.4.13 setParameter.....	31
3.4.14 getTemplateQuality.....	32
3.4.15 getTemplateLength.....	32
3.4.16 convertTemplate.....	33
3.4.17 getFPLimitCount.....	33
3.5 Instance Codes (for details, see Demo-Android Studio).....	33
4 . ZKFinger SDK Flowchart.....	40
4.1 ZKFinger SDK Algorithm Flowchart.....	40
4.2 One-Enrollment Process.....	42
4.3 Multi-Enrollment Process.....	43
Appendix.....	45

1. Overview

ZKFinger SDK is a set of application programming interfaces (APIs) provided by ZKTeco for development engineers. It is capable of managing ZKTeco fingerprint sensors in a unified manner. It can be used by developers in different classes to develop applications for controlling ZKTeco Android-based devices.

ZKFinger SDK supports the following functions:

Fingerprint sensors: ZKFinger SDK allows performing operations on fingerprint sensors, including initialization, startup, and shutdown.

Fingerprint algorithm: The ZKFinger SDK supports 1:1 and 1:N fingerprint comparison and so on.

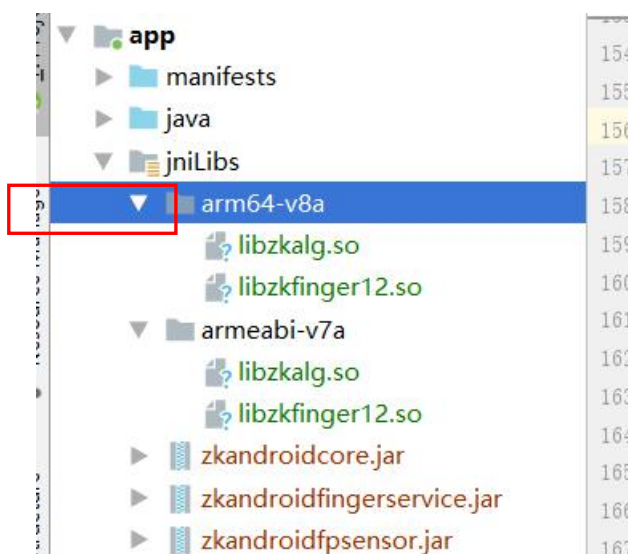
2. Development Environment Setup

2.1 Importing zkandroid.jar/zkandroidcore.jar/zkandroidfingerservice.jar

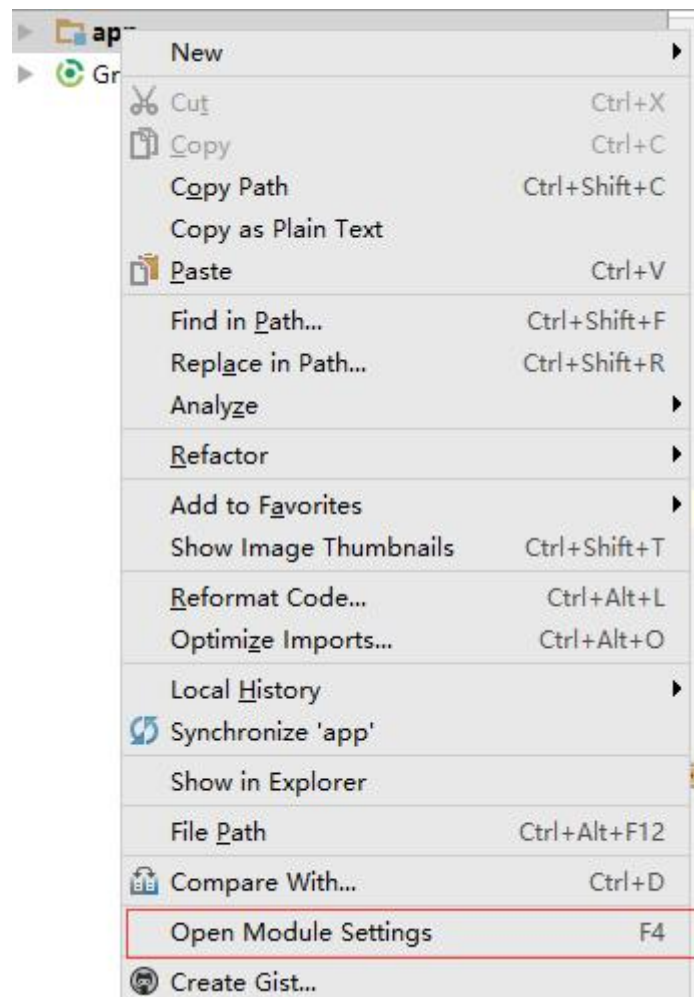
Open the ZKDevice SDK folder, and import

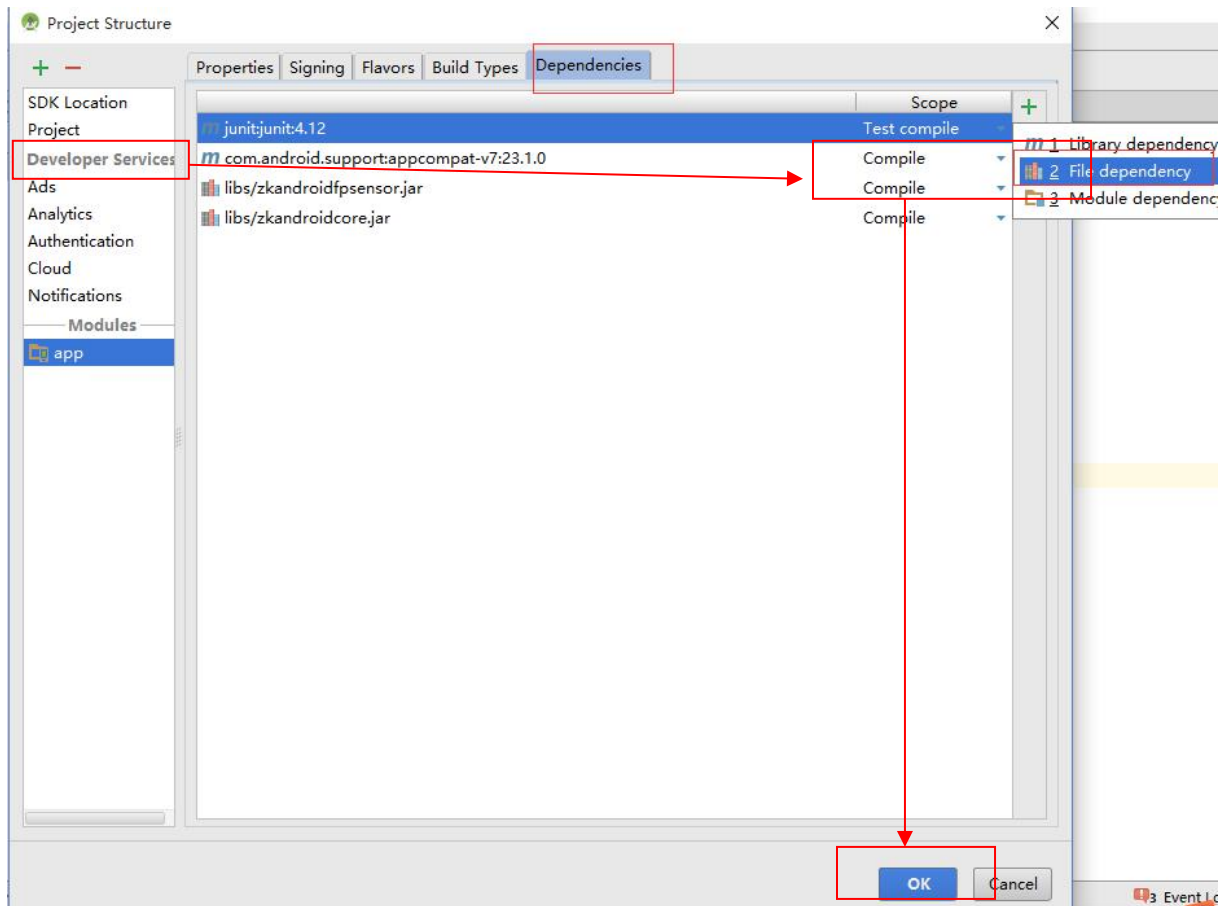
zkandroidfpsensor.jar/zkandroidcore.jar/zkandroidfingerservice.jar in the Device directory to the application development tool (the following uses Android Studio as an example).

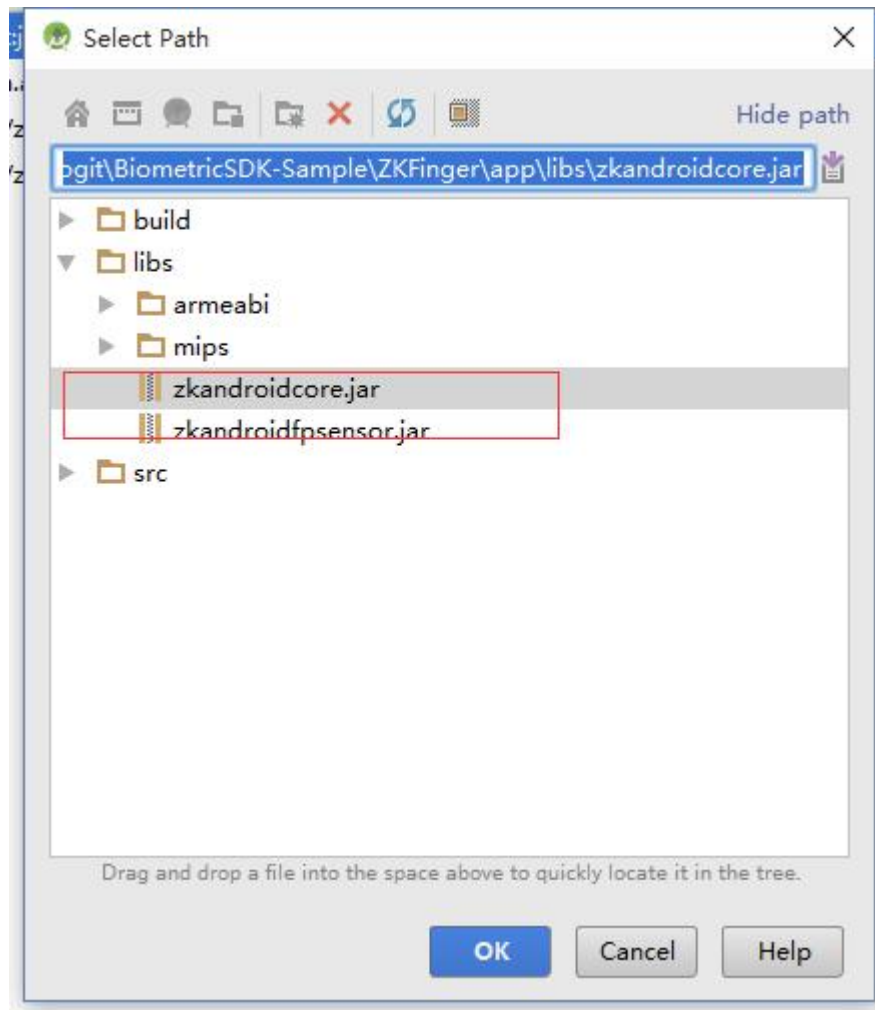
Step 1: Copy zkandroidfpsensor.jar ,zkandroidcore.jar and zkandroidfingerservice.jar into the app/lib directory.



Step 2: Right-click a project and choose Open Modules Setting from the shortcut menu. In the window displayed, click on Dependencies tab, click +, and select File dependency. In the Select Path dialog box, select zkandroidfpsensor.jar/zkandroidcore.jar in the libs directory, click OK to import zkandroidfpsensor.jar/zkandroidcore.jar.







3. ZKFinger SDK

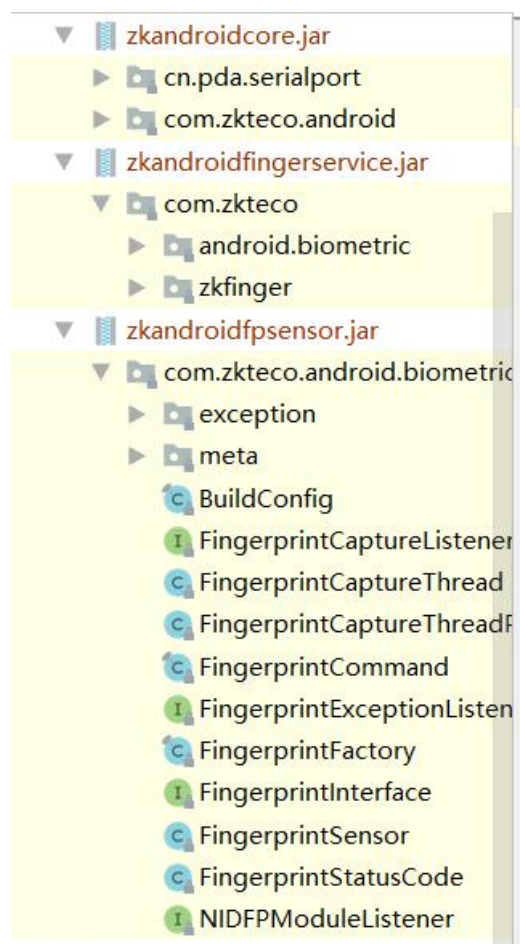
ZKFinger SDK abstracts function modules as classes. Users can call methods in classes to complete operations on the underlying hardware as well as handling of the fingerprint algorithm.

ZKFinger SDK includes the fingerprint sensor class, control device class, constant class, exception handling class, and algorithm handling class. The following table lists the types of the classes.

Class Name	Device Type
<code>com.zkteco.android.biometric.module.fingerprint.FingerprintSensor</code>	Fingerprint sensor class
<code>com.zkteco.android.biometric.module.fingerprint.BaseStatusCode</code>	Error code
<code>com.zkteco.android.biometric.module.fingerprint.FingerprintCaptureListener</code>	Callback interface class

com.zkteco.android.biometric.module.fingerprint. exception.FingerprintSensorException	Exception handling class
com.zkteco.android.biometric.core.utils.LogHelpe r	Log tool class
com.zkteco.android.biometric.core.utils.ToolUtils	Auxiliary tool class
com.zkteco.android.biometric.module.fingerprintreader. ZKFingerService	Algorithm handling class

The figures below show the structure of the SDK package.



3.1 FingerprintSensor.class

FingerprintSensor.class is mainly used to control fingerprint sensors, for example, initialize the devices, start the fingerprint sensors, capture fingerprint images, and shut down the devices. For the calling process, see the flowchart of the fingerprint sensor class.

3.1.1 open

[Interface]

```
public void open(int index)
```

[Purpose]

This interface is used to start a fingerprint sensor.

[Parameter Description]

index

Device index number. The value is determined by the total number of connected sensors.

Example:

When the total number of connected sensors is 1, the index value is 0.

When the total number of connected sensors is 2, the index value is 0 or 1.

... so on.

[Return Value]

None.

[Note]

An error code is thrown out through an exception.

Typically, the scenario uses a single device, with index passing 0

3.1.2 close

[Interface]

```
public void close(int index)
```

[Purpose]

This interface is used to shut down a fingerprint sensor.

[Parameter Description]

index

Device index number. The value is determined by the total number of connected sensors.

Example:

When the total number of connected sensors is 1, the index value is 0.

When the total number of connected sensors is 2, the index value is 0 or 1.

... so on.

[Return Value]

None.

[Note]

An error code is thrown out through an exception.

3.1.3 retrieveLastImage

[Interface]

```
public int retrieveLastImage(int index, byte[] buffer, int[] imageAttributes)
```

[Purpose]

This interface is used to capture the previous fingerprint image. (After starCapture, the background thread automatically calls the function, when the finger press active call **FingerprintCaptureListener.CaptureOK** return image and character, do not need to take the initiative to call the function.)

[Parameter Description]

index

Device index number. The value is determined by the total number of connected sensors.

Example:

When the total number of connected sensors is 1, the index value is 0.

When the total number of connected sensors is 2, the index value is 0 or 1.

... so on.

buffer

Array for storing returned image data.

It is recommended that the size of the buffer array to be set to 480x640.

The image width and height are acquired after a fingerprint image is successfully captured.

imageAttributes

Image width and height array.

imageAttributes[0] is used for storing the image width.

imageAttributes[1] is used for storing the image height.

[Return Value]

> 0 Image data length

[Note]

The previous fingerprint image is captured after acquireTemplate is called successfully.

An error code is thrown out through an exception.

3.1.4 acquireTemplate

[Interface]

```
public int acquireTemplate (int index, byte[] buffer)
```

[Purpose]

This interface is used to acquire a fingerprint template.(After starCapture, the background thread automatically calls the function, when the finger press active call **FingerprintCaptureListener. CaptureOK** return image and character, do not need to take the initiative to call the function.)

[Parameter Description]

index

Device index number. The value is determined by the total number of connected sensors.

Example:

When the total number of connected sensors is 1, the index value is 0.

When the total number of connected sensors is 2, the index value is 0 or 1.

... so on.

buffer

Array for storing returned fingerprint template data.

[Return Value]

> 0 Fingerprint template data length

[Note]

An error code is thrown out through an exception.

3.1.5 getLastTempLen

[Interface]

```
public int getLastTempLen()
```

[Purpose]

This interface is used to get the template length(while captureOK).

[Parameter Description]

[Return Value]

The length of template

[Note]

Current fingerprint template data length. can be get **While FingerprintCaptureListener.captureOK.**

3.1.6 setFingerprintCaptureListener

[Interface]

```
public void setFingerprintCaptureListener(int index, FingerprintCaptureListener listener)
```

[Purpose]

This interface is used to set a callback function.

[Parameter Description]

index

Device index number. The value is determined by the total number of connected sensors.

Example:

When the total number of connected sensors is 1, the index value is 0.

When the total number of connected sensors is 2, the index value is 0 or 1.

... so on.

FingerprintCaptureListener

Callback interface

[Return Value]

[Note]

Images are captured asynchronously. For details, see the demo.

3.1.7 *setFingerprintCaptureMode*

[Interface]

```
public void setFingerprintCaptureMode(int index, int mode)
```

[Purpose]

This interface is used to set the capture mode (asynchronous).

[Parameter Description]

Index

Device index number. The value is determined by the total number of connected sensors.

Example:

When the total number of connected sensors is 1, the index value is 0.

When the total number of connected sensors is 2, the index value is 0 or 1.

... so on.

mode

The value 0 indicates that only fingerprint templates are captured.(It is recommended to take only the template when the serial port is connected. It takes more than 10s)

The value 1 indicates that fingerprint templates and fingerprint images are captured.

[Return Value]

[Note]

This interface is called after startCapture is called successfully.

3.1.8 startCapture

[Interface]

```
public void startCapture(int index)
```

[Purpose]

This interface is used to start capturing images asynchronously.

[Parameter Description]

index

Device index number. The value is determined by the total number of connected sensors.

Example:

When the total number of connected sensors is 1, the index value is 0.

When the total number of connected sensors is 2, the index value is 0 or 1.

... so on.

[Return Value]

[Note]

Images are captured asynchronously and the callback interface set by `setFingerprintCaptureListener` returns the fingerprint image template. For details, see the demo.

3.1.9 stopCapture

[Interface]

```
public void stopCapture (int index)
```

[Purpose]

This interface is used to stop capturing images asynchronously.

[Parameter Description]

index

Device index number. The value is determined by the total number of connected sensors.

Example:

When the total number of connected sensors is 1, the index value is 0.

When the total number of connected sensors is 2, the index value is 0 or 1.

... so on.

[Return Value]

[Note]

Asynchronous capture of images is stopped.

3.1.10 Destroy

[Interface]

```
public void destroy()
```

[Purpose]

This interface is used to destroy resources.

[Parameter Description]

None

[Return Value]

[Note]

3.1.11 getFirmwareVersion

[Interface]

```
public String getFirmwareVersion()
```

[Purpose]

Get firmware version

[Parameter Description]

None

[Return Value]

Firmware Version

[Note]

When open it successfully ,firmware version will display.

3.1.12 getSerialNumber

[Interface]

```
public String getSerialNumber()
```

[Purpose]

Get serial number

[Parameter Description]

None

[Return Value]

Serial number

[Note]

When open it successfully ,firmware version will display.

3.1.13 rebootDevice

[Interface]

```
public void rebootDevice(int index)
```

[Purpose]

Reboot the device

[Parameter Description]

index

referring to open part

[Return Value]

None

[Note]

When open it successfully ,the function can be called.

3.1.14 rebootDeviceEx

[Interface]

```
public void rebootDeviceEx(int index)
```

[Purpose]

Reboot the device

[Parameter Description]

index

referring to open part

[Return Value]

None

[Note]

No need to open first.

3.1.15 version

[Interface]

public String version()

[Purpose]

Get SDK version

[Parameter Description]

None

[Return Value]

SDK Version

[Note]

3.2 [FingerprintCaptureListener.class](#)

FingerprintCaptureListener.class is an asynchronous image capture interface class. For the calling method, see the Demo.

3.2.1 [captureOK](#)

[Interface]

void captureOK(int captureMode, byte[] imageBuffer, int[]imageAttributes, byte[] templateBuffer);

[Purpose]

This interface indicates capture success.

[Parameter Description]

mode

The value 0 indicates that only fingerprint templates are captured.

The value 1 indicates that fingerprint templates and fingerprint images are captured.

imageBuffer

Array for storing returned image data (it refers to the position null when mode is 0.).

The image width and height are acquired after a fingerprint image is successfully captured.

imageAttributes

Image width and height array (it refers to the position null when mode is 0).

imageAttributes[0] is used for storing the image width.

imageAttributes[1] is used for storing the image height.

templateBuffer

Array for storing returned fingerprint template data.

[Return Value]

[Note]

3.2.2 captureError

[Interface]

void captureError(FingerprintSensorException e)

[Purpose]

This interface indicates a capture failure.

[Parameter Description]

e: abnormal object (see 3.3).

[Return Value]

[Note]

3.3FingerprintSensorException.class

FingerprintSensorException.class is an exception handling class.

3.3.1 getErrorCode

[Interface]

public int getErrorCode()

[Purpose]

This interface is used to get an error code.

[Parameter Description]

[Return Value]

Error code

[Note]

3.3.2 getInternalErrorCode

[Interface]

```
public int getInternalErrorCode ()
```

[Purpose]

This interface is used to get an internal (underlying) error code.

[Parameter Description]

[Return Value]

Error code

[Note]

3.4 ZKFingerService.class

ZKFingerService.class is a fingerprint algorithm handling class.

3.4.1 init

[Interface]

```
public static native int init ()
```

[Purpose]

This interface is used to initialize the fingerprint identification engine.

[Parameter Description]

[Return Value]

0 Succeeded.

Otherwise, an error code is returned. For the description of error codes, refer .

[Note]

The fingerprint identification engine is the fingerprint identification algorithm library.

3.4.2 free

[Interface]

```
public static native int free()
```

[Purpose]

This interface is used to shut down the fingerprint identification engine and clear all fingerprint templates stored in the buffer.

[Parameter Description]

None.

[Return Value]

0 Succeeded.

Otherwise, an error code is returned. For the description of error codes, refer .

3.4.3 verify**[Function]**

```
static public int verify(byte[] temp1, byte[] temp2)
```

[Purpose]

This function is used to compare two fingerprint templates.

[Parameter Description]

temp1

The first fingerprint template data array

temp2

The second fingerprint template data array

[Return Value]

Fingerprint template matching score, with the value ranging from 0 to 1000.

Suggestion: When the return value is larger than the preset threshold or default value (50), it is considered that two fingerprint templates are matched successfully.

[Note]

For the description of error codes, see Appendix.

3.4.4 verifyId

[Function]

```
static public int verifyId( byte[] temp, String id)
```

[Purpose]

This function is used to match a fingerprint template with the fingerprint template of a specified ID in the buffer (1:1 comparison). A matching score will be returned.

[Parameter Description]

temp

Fingerprint template data array

id

ID of the target fingerprint template for comparison. The ID is a string, with the length not longer than 20 bytes.

Example: String id = "ZKTeco_20_01"

[Return Value]

Fingerprint template matching score, with the value ranging from 0 to 1000.

When the return value is larger than the preset threshold or default value (50), two fingerprint templates are matched successfully.

3.4.5 identify

[Function]

```
static public int identify(byte[] temp, byte []idstr, int threshold, int count)
```

[Purpose]

This function is used to match fingerprint template data with fingerprint template data in the buffer (that is, 1:N comparison). It returns the count of matching scores that are higher than the threshold. If the actual count is smaller than the preset count, the actual count is returned. For example, if count is set to 2, the return value may be 0, 1, or 2.

[Parameter Description]

temp

Fingerprint template data array

idstr

Array of the IDs and scores of fingerprint templates that are matched successfully.

It is recommended that the array size be set to 50*count.

Multiple results are sorted by matching score in descending order. Each result contains the ID and matching score. IDs and matching scores are separated by /t and results are separated by /n.

Example: String resultStrs = new String(idstr);

If the return value of resultStrs is 1001\t'95'\n'1002\t'85'\n'1003\t75, three comparison results are returned:

Result 1: ID: 1001, score: 95

Result 2: ID: 1002, score: 85

Result 3: ID: 1003, score: 75

For details about codes, see [Example Code]

threshold

Matching threshold, with the value ranging from 0 to 1000. Note: This value will replace the preset threshold.

Matching threshold determines the level of similarity between two fingerprint templates. A higher matching threshold indicates a stricter requirement for the similarity. When two fingerprint templates are compared and the return value is greater than the preset threshold, the comparison is successful. Otherwise, the comparison fails. Flexible adjustment can be made based on conditions in specific applications.

Matching threshold description

False Rejection Rate	False Accept Rate	Matching Threshold	
		1:N	1:1
High	Low	80	60
Intermediate	Intermediate	75	55
Low	High	70	50

count

Expected number of returned results. The recommended value is 1.

Example:

When threshold is set to 70 and count is set to 3, if only two fingerprint templates meet the matching threshold during comparison, the return value is 2.

[Return Value]

Succeeded The actual comparison result count is returned. The return value may be smaller than value of count. The value 1 indicates that one fingerprint template is matched successfully. The value 0 indicates that no fingerprint template is matched successfully.

Otherwise, an error code is returned.

[Note]

For the description of error codes, see Appendix.

[Example Code]

```
int ret = 0;

byte[] idstr = new byte[50];

ret = ZKFingerServer.identify(tempIdentify, idstr, 70, 1);

if( ret == 1 ){

    String resultString = new String(idstr);

    textView.setText("id: " + resultString.split("\t")[0] + ", score: " +
resultString.split("\t")[1] );

}else{

    textView.setText("Please try again!");

}
```

3.4.6 merge

[Function]

```
static public int merge(byte[] temp1, byte[] temp2, byte[] temp3, byte[] temp)
```

[Purpose]

This function is used to merge three fingerprint templates as an enrolled template.

[Parameter Description]

temp1

Fingerprint template 1 to be enrolled

temp2

Fingerprint template 2 to be enrolled

temp3

Fingerprint template 3 to be enrolled

temp

Output enrolled fingerprint template

[Return Value]

>0 the length of enrolled fingerprint template

Otherwise, an error code is returned.

[Note]

For the description of error codes, see Appendix.

3.4.7 save

[Function]

```
static public int save ( byte[] temp, String id )
```

[Purpose]

This function is used to store fingerprint template data to the buffer. Users can save required fingerprint data in the database to the buffer by invoking the save method.

[Note]

Ensure that the fingerprint algorithm library is initialized successfully when the save method is called.

[Parameter Description]

temp

Transferred fingerprint template data array

id

Fingerprint template ID

The ID is a string, with the length not longer than 20 bytes.

Example: String id = "ZKTeco_20_01"

[Return Value]

0 Succeeded

Otherwise, an error code is returned.

[Note]

For the description of error codes, see Appendix.

3.4.8 get

[Function]

```
static public int get ( byte[] temp, String id )
```

[Purpose]

This function is used to acquire the fingerprint template data of a specified ID in the buffer.

[Parameter Description]

temp

Fingerprint template data array of a specified ID. It is recommended that the array size be set to 1024*3, to ensure that there is sufficient space for storing fingerprint template data.

id

ID of the fingerprint template to be acquired

The ID is a string, with the length not longer than 20 bytes.

Example: String id = "ZKTeco_20_01"

[Return Value]

>0 Succeeded. The value is the fingerprint template length.

Otherwise, an error code is returned.

[Note]

For the description of error codes, see Appendix.

3.4.9 del

[Function]

```
static public int del(String id)
```

[Purpose]

This function is used to delete an enrolled fingerprint template from the memory.

[Parameter Description]

id

ID of the fingerprint template to be deleted

The ID is a string, with the length not longer than 20 bytes.

Example: String id = "ZKTeco_20_01"

[Return Value]

0 Succeeded

Otherwise, an error code is returned.

[Note]

For the description of error codes, see Appendix.

3.4.10 clear

[Function]

```
static public int clear ()
```

[Purpose]

This function is used to clear all fingerprint templates from the buffer.

[Parameter Description]

None.

[Return Value]

0 Succeeded

Otherwise, an error code is returned.

[Note]

For the description of error codes, see Appendix.

3.4.11 count

[Function]

```
static public int count()
```

[Purpose]

This function is used to acquire the number of fingerprint templates stored in the buffer.

[Parameter Description]

None.

[Return Value]

The number of fingerprint templates stored in the buffer is returned.

3.4.12 version

[Function]

```
static public void version(byte[] version)
```

[Purpose]

Get the algorithm version

[Parameter Description]

version

Return the algorithm version

[Return Value]

3.4.13 setParameter

[Function]

```
static public int setParameter(int opCode, int value)
```

[Purpose]

Set the algorithm version

[Parameter Description]

opCode

Option Code(1 means 1:1 threshold)

vaule

value(opCode=1 means 1:1 threshold default value is 50)

[Return Value]

0 means succeed

Others means failure

[Note]

For non-developer only set OpCode=1 is allowed.

3.4.14 getTemplateQuality

[Function]

static public int getTemplateQuality(byte[] template)

[Purpose]

Get Template Quality

[Parameter Description]

template

FP Template

[Return Value]

Template Quality

3.4.15 getTemplateLength

[Function]

static public int getTemplateLength(byte[] template)

[Purpose]

Get FP Template Length

[Parameter Description]

template

FP Template

[Return Value]

FP template data effective length

3.4.16 convertTemplate

[Function]

static public int convertTemplate(byte[] inTemp, byte[] outTemp)

[Purpose]

Template transformation(12.0->10.0)

[Parameter Description]

inTemp

12.0 template

outTemp

10.0 template

[Return Value]

> 0 means succeed, and the return value is 10.0 template data length

3.4.17 getFPLimitCount

[Function]

static public int getFPLimitCount()

[Purpose]

Get 1:N FP limit count

[Parameter Description]

None

[Return Value]

1:N FP limit count

3.5 Instance Codes (for details, see Demo-Android Studio)

```
package com.zkteco.slk20mdemo;
import android.graphics.Bitmap;
import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.widget.ImageView;
import android.widget.TextView;
```

```
import androidx.appcompat.app.AppCompatActivity;

import com.zkteco.android.biometric.core.device.ParameterHelper;
import com.zkteco.android.biometric.core.device.TransportType;
import com.zkteco.android.biometric.core.utils.LogHelper;
import com.zkteco.android.biometric.core.utils.ToolUtils;
import com.zkteco.android.biometric.module.fingerprint.FingerprintCaptureListener;
import com.zkteco.android.biometric.module.fingerprint.FingerprintFactory;
import com.zkteco.android.biometric.module.fingerprint.FingerprintSensor;
import com.zkteco.android.biometric.module.fingerprint.exception.FingerprintSensorException;
import com.zkteco.android.biometric.module.fingerprintreader.ZKFingerService;

import java.util.HashMap;
import java.util.Map;

public class MainActivity extends AppCompatActivity {
    private static final int VID = 6997; //zkteco device VID 6997
    private static final int PID = 289;
    private boolean mbStart = false;
    private TextView txtView = null;
    private ImageView imageView = null;
    private int index = 0;
    private final static int ENROLL_COUNT = 3;
    private int enroll_index = 0;
    private byte[][] regtemparray = new byte[3][2048]; //register template buffer array
    private boolean bRegister = false;
    private static FingerprintSensor fingerprintSensor = null;

    private void initUI()
    {
        txtView = (TextView)findViewById(R.id.txtView);
        imageView = (ImageView)findViewById(R.id.imageView);
    }

    private void startFingerprintSensor() {
        if (null != fingerprintSensor)
        {
            FingerprintFactory.destroy(fingerprintSensor);
        }
        // Define output log level
        LogHelper.setLevel(Log.ASSERT);

        // Start fingerprint sensor
    }
}
```

```
Map fingerprintParams = new HashMap();
//set vid
fingerprintParams.put(ParameterHelper.PARAM_KEY_VID, VID);
//set pid
fingerprintParams.put(ParameterHelper.PARAM_KEY_PID, PID);
fingerprintSensor = FingerprintFactory.createFingerprintSensor(getApplicationContext(), TransportType.USB, fingerprintParams);
}

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    initUI();
}

@Override
protected void onDestroy()
{
    super.onDestroy();
    if (null != fingerprintSensor) {
        FingerprintFactory.destroy(fingerprintSensor);
    }
}

private void setTextResult(String strResult)
{
    final String flStrResult = strResult;
    runOnUiThread(new Runnable() {
        public void run() {
            txtView.setText(flStrResult);
        }
    });
}

void doRegister(byte[] template, int templateLen)
{
    byte[] bufids = new byte[256];
    int ret = ZKFingerService.identify(template, bufids, 70, 1);
    if (ret > 0)
    {
        String strRes[] = new String(bufids).split("\t");
        setTextResult("the finger already enroll by " + strRes[0] + ",cancel enroll");
    }
}
```

```
bRegister = false;
enroll_index = 0;
return;
}
if (enroll_index > 0 && (ret = ZKFingerService.verify(regtemparray[enroll_index-1], template)) <= 0)
{
    setTextResult("please press the same finger 3 times for the enrollment, cancel enroll, socre=" + ret);
    bRegister = false;
    enroll_index = 0;
    return;
}
System.arraycopy(template, 0, regtemparray[enroll_index], 0, 2048);
enroll_index++;
if (enroll_index == ENROLL_COUNT) {
    byte[] regTemp = new byte[2048];
    int regTemplateLen = 0;
    if (0 < (regTemplateLen = ZKFingerService.merge(regtemparray[0], regtemparray[1], regtemparray
[2], regTemp))) {
        int retVal = 0;
        retVal = ZKFingerService.save(regTemp, "uid_" + (index++));
        if (0 == retVal)
        {
            setTextResult("enroll succ");
        }
        else
        {
            setTextResult("enroll fail, add template fail!");
        }
    } else {
        setTextResult("enroll fail");
    }
    bRegister = false;
} else {
    setTextResult("You need to press the " + (3 - enroll_index) + "time fingerprint");
}
}

void doIdentify(byte[] template, int templateLen)
{
    byte[] bufids = new byte[256];
    int ret = ZKFingerService.identify(template, bufids, 70, 1);
    if (ret > 0) {
        String strRes[] = new String(bufids).split("\t");
        setTextResult("identify succ, userid:" + strRes[0].trim() + ", score:" + strRes[1].trim());
    }
}
```

```
    } else {
        setResult("identify fail, ret=" + ret);
    }
}

private void openDevice()
{
    startFingerprintSensor();
    try {
        bRegister = false;
        enroll_index = 0;
        //init zkfinger10
        int retVal = ZKFingerService.init();
        if (0 != retVal)
        {
            setResult("init zkfinger10 failed, ret=" + retVal);
            return;
        }
        fingerprintSensor.open(0);
        FingerprintCaptureListener fingerprintCaptureListener = new FingerprintCaptureListener() {
            @Override
            public void captureOK(int mode, byte[] rawImage, int[] attribs, byte[] template) {
                if (mode == 1) //capture template and image
                {
                    if (null != rawImage)
                    {
                        final int width = attribs[0];
                        final int height = attribs[1];
                        final Bitmap bitmap = ToolUtils.renderCroppedGreyScaleBitmap(rawImage, width, height);
                        runOnUiThread(new Runnable() {
                            public void run() {
                                imageView.setImageBitmap(bitmap);
                            }
                        });
                    }
                }
            }
        };
        if (bRegister)
        {
            doRegister(template, fingerprintSensor.getLastTempLen());
        }
        else
        {
            doIdentify(template, fingerprintSensor.getLastTempLen());
        }
    }
}
```

```
    }

    @Override
    public void captureError(FingerprintSensorException e) {

    }
};
fingerprintSensor.setFingerprintCaptureListener(0, fingerprintCaptureListener);
fingerprintSensor.startCapture(0);
setTextResult("begin success, firmwareversion:" + fingerprintSensor.getFirmwareVersion() + ",serial
=" + fingerprintSensor.getSerialNumber());
} catch (FingerprintSensorException e) {
    e.printStackTrace();
    try {
        fingerprintSensor.rebootDeviceEx(0);
    } catch (FingerprintSensorException ex) {
        //ex.printStackTrace();
    }
    setTextResult("open device failed!");
}
}

public void onBnStart(View view)
{
    openDevice();
}

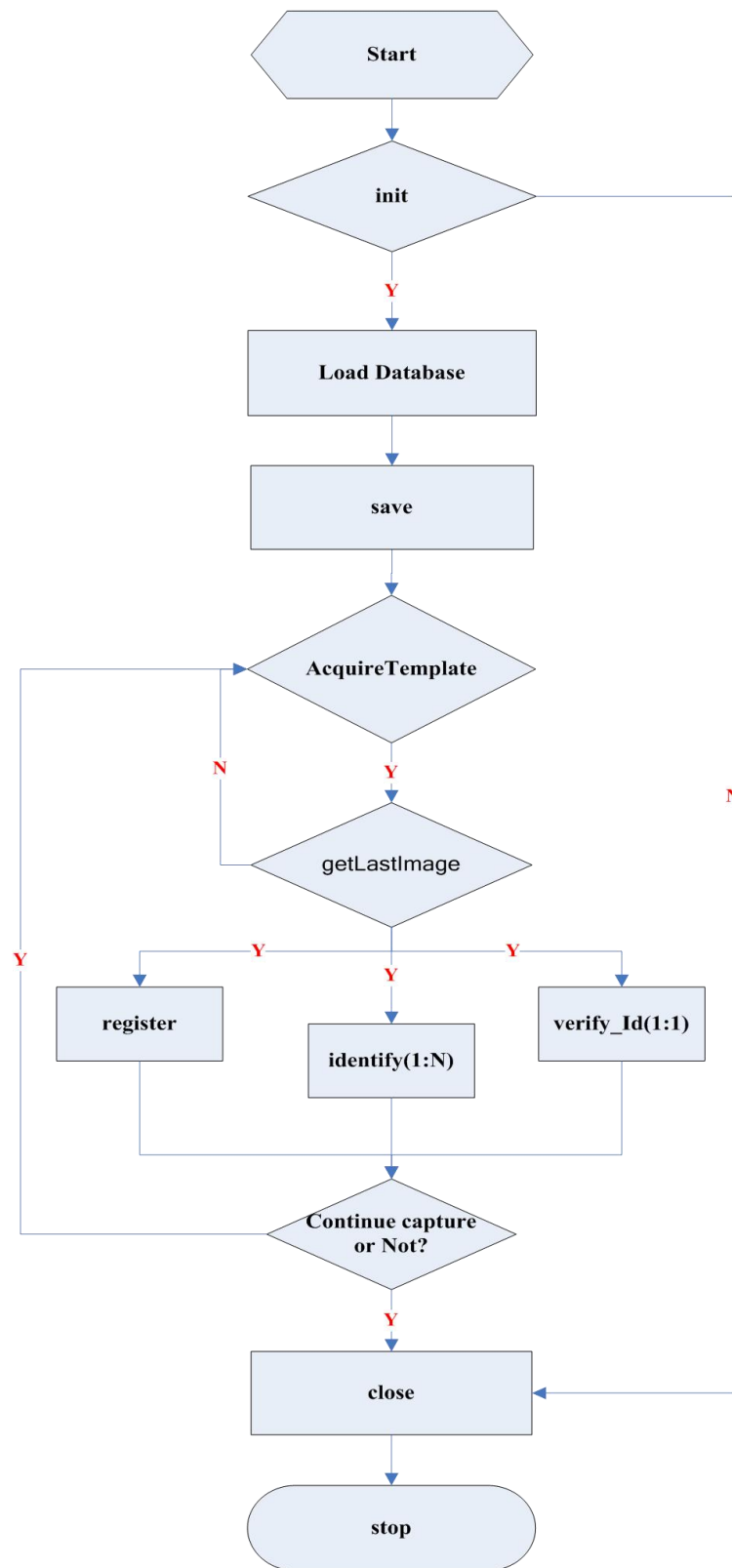
public void onBnEnroll(View view)
{
    bRegister = true;
    enroll_index = 0;
    setTextResult("Please press your finger 3 times(same finger.)");
}

private void closeDevice()
{
    if (mbStart)
    {
        try {
            fingerprintSensor.stopCapture(0);
            fingerprintSensor.close(0);
        } catch (FingerprintSensorException e) {
            //e.printStackTrace();
        }
    }
}
```

```
    }  
    mbStart = false;  
}  
  
public void onBnStop(View view)  
{  
    closeDevice();  
    setResult("device closed!");  
}  
}
```

4 . ZKFinger SDK Flowchart

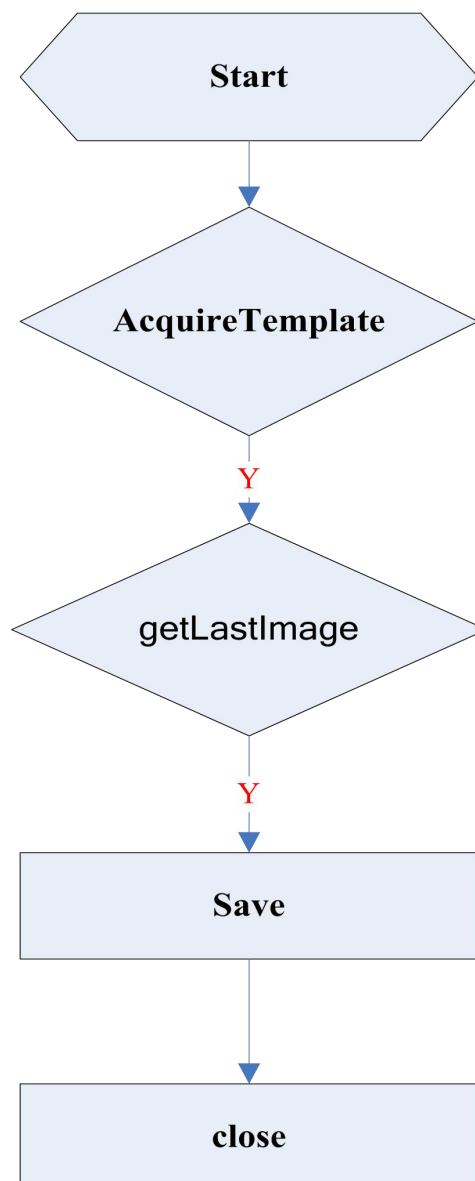
4.1 ZKFinger SDK Algorithm Flowchart



Notes:

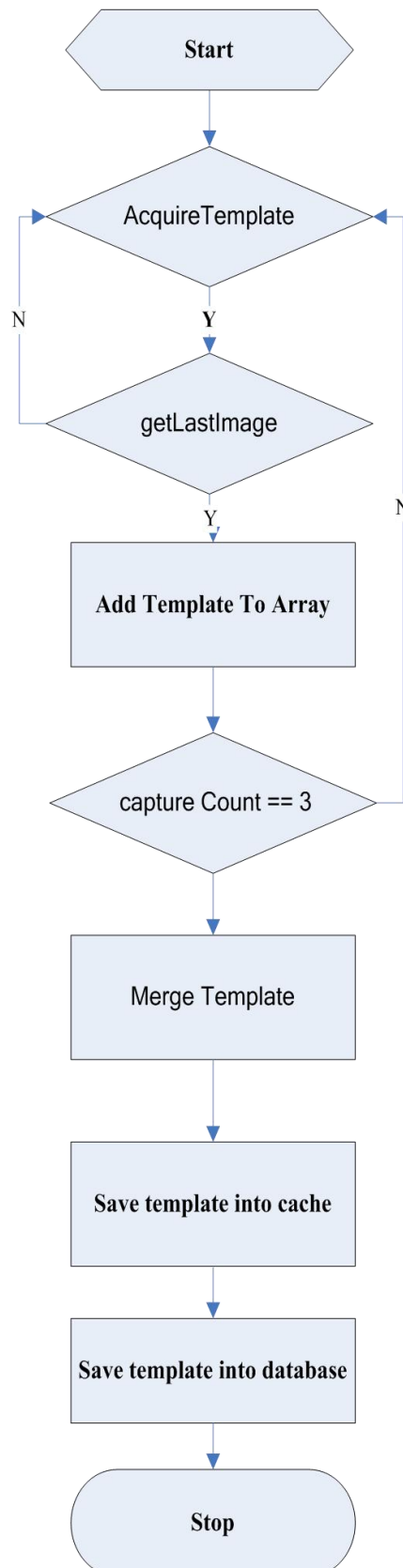
- The fingerprint algorithm library does not support the database storage function. It temporarily stores data in the buffer. Ensure that data is synchronized to the database during fingerprint enrollment and deletion.
- Before an application is run, required fingerprint data (including the ID and fingerprint templates) need to be extracted from the database and then the save interface is called to load the data to the fingerprint algorithm library buffer.
- When an application starts running, required fingerprint data (including the ID and fingerprint templates) need to be searched out in the user database and then the save interface is called to load the data to the fingerprint algorithm library buffer.

4.2 One-Enrollment Process



4.3 Multi-Enrollment Process

Note: This Flowchart Uses Three Times of Enrollment as an Example.



- Step 1 Call `acquireTemplate` to acquire fingerprint template data (`temp*`).when acquiring “temp1”
Consecutive three fingerprint templates captured in step 1 are returned.
- Step 2 Call the merge interface to combine `temp1`, `temp2`, and `temp3` as `enrolltemp`.
- Step 3 Call the save interface to save the fingerprint template `enrolltemp` to the buffer and database.
- Step 4 Repeat the preceding steps if additional fingerprint enrollment is required.

Appendix

Error code	Description
-5000	Failed to find the fingerprint template of the specified ID.
-5002	Parameter error.
-5003	Fingerprint template error.
-5004	Method error.
-1116	The algorithm is not initialized
-1124	Unspecified internal error occurred.
-1128	Exceeded database user limit.
-1135	Invalid template or unsupported template format.